Outils de résolution de problèmes d’optimisation

Année académique 2026 - 2027
Université : Université d’Abomey-Calavi
Établissement : Institut de Formation et de Recherche en Informatique (IFRI)
Grade : Licence 3 en Intelligence Artificielle
Chargé du cours : Dr Ing. (MC) Houndji Vinasetan Ratheil, https://ratheil.info

Projet 2026-2027 / Planification automatique de soutenances

Sujet étudiant · L3 Intelligence artificielle · Travail en 8 groupes

Votre groupe doit développer un programme qui compose les jurys et construit un calendrier de soutenances valide, puis en améliore la qualité. Vous réaliserez une méthode de référence simple, au moins une méthode d’optimisation et une comparaison reproductible sur les 30 instances fournies.

Le projet reprend la problématique de composition et de planification des jurys étudiée dans le travail de Jean Eudes. Le jeu de données et les outils de ce dossier constituent le support du présent projet.

Démarche attendue

- Modéliser les décisions et les contraintes, puis vérifier la conformité des plannings produits.
- Comparer une méthode de référence à au moins une méthode d’optimisation sur les 30 instances, avec un protocole reproductible.
- Expliquer les compromis entre critères, documenter les résultats et présenter des plannings commentés.

Remise par groupe

Le travail comprend le code, une notice de reproduction, les résultats expérimentaux, un rapport et un support de restitution. Les contraintes, les critères de qualité, le protocole, les livrables détaillés et le barème figurent dans les pages suivantes.

1. Comprendre le problème

Une instance décrit des étudiants, des enseignants, des salles et un horizon découpé en jours et en créneaux. Pour chaque séance, vous choisissez le président, l'examinateur, le jour et le créneau. Les pré-soutenances se déroulent en ligne ; seules les finales reçoivent une salle physique. L'encadrant de chaque étudiant est déjà fixé et assure également le rôle de rapporteur. Deux périodes communes à tous les étudiants fixent les jours de pré-soutenance puis les jours de finale. Chaque passage occupe exactement un créneau, dans les deux phases.

Phase à planifier	Composition et participation	Modalité
Pré-soutenance	Président et examinateur, avec l'étudiant, à distance. Une séance pour chaque étudiant.	En ligne Sans salle
Soutenance finale	Même président et même examinateur, plus l'encadrant agissant comme rapporteur, avec l'étudiant. Une séance si eligible_final vaut true.	En salle

Les membres d'un même jury sont distincts. L'admissibilité à la soutenance finale est une donnée de l'instance : votre programme ne décide pas du passage d'un étudiant. Un étudiant dont eligible_final vaut false a uniquement une pré-soutenance.

Le jeu de données

Les 30 instances sont synthétiques. Elles sont réparties en trois tailles, avec 10 instances par taille. Les identifiants des personnes n'ont pas de signification métier et ne doivent pas orienter les choix algorithmiques.

Taille	Étudiants par instance	Instances
Petite	12 à 20	10
Moyenne	40 à 58	10
Grande	100 à 136	10

Le nombre de séances à planifier est le nombre total d'étudiants, augmenté du nombre d'étudiants admissibles en finale. Les caractéristiques exactes de chaque fichier figurent dans data/manifest.json.

Points à vérifier tout au long du projet

- Les pré-soutenances sont en ligne sans salle ; périodes communes, créneau unique et continuité des jurys sont respectés.
- Tous les fichiers de solution annoncés comme valides passent le validateur ; leurs métriques proviennent du contrôle final.
- Les résultats couvrent les 30 instances et toutes les répétitions prévues, y compris les échecs et les limites de temps.
- Un autre groupe peut retrouver vos résultats à partir des commandes, versions, graines et configurations fournies.

2. Respecter les contraintes impératives

Une solution n’est recevable que si toutes les contraintes ci-dessous sont respectées. Aucun gain sur un critère de qualité ne peut compenser une violation.

Couverture et composition

- Planifier exactement une pré-soutenance par étudiant. Ajouter exactement une finale pour chaque étudiant admissible, et aucune finale pour les autres. Aucun étudiant ni aucune phase ne doit être dupliqué.
- Attribuer le rôle de président uniquement à un enseignant habilité à ce rôle, et celui d’examinateur uniquement à un enseignant habilité à examiner. L’examinateur doit couvrir le domaine de l’étudiant ; le président peut appartenir à un autre domaine.
- En pré-soutenance, président et examinateur sont distincts. En finale, président, examinateur et encadrant sont tous distincts. L’encadrant est le rapporteur et ne constitue donc pas un quatrième enseignant. Il n’a besoin d’aucune habilitation supplémentaire ni d’un domaine particulier pour assister à la finale dans ce rôle.

Calendrier et ressources

- Planifier chaque passage sur exactement un créneau, en pré-soutenance comme en finale. Une séance affectée au créneau s occupe seulement ce créneau, soit l’intervalle [s, s + 1). Le jour et le créneau doivent appartenir à l’horizon défini.
- Vérifier la disponibilité de l’étudiant et de chaque enseignant présent au jour et au créneau choisis, dans les deux phases. Vérifier aussi la disponibilité de la salle pour une finale. Un jour ou un créneau absent de la disponibilité signifie une indisponibilité.
- Interdire tout chevauchement pour un même étudiant ou enseignant présent, dans les deux phases, et pour une même salle lors des finales. Deux séances peuvent se suivre immédiatement. L’encadrant assiste à la finale. En pré-soutenance, il intervient seulement s’il est choisi comme président ou examinateur, selon les règles du jury.
- Respecter le plafond journalier de séances de chaque enseignant dans les deux phases. Pour les finales, respecter aussi le plafond journalier de la salle. Les pré-soutenances n’utilisent aucune salle et ne sont soumises à aucune disponibilité ni limite de simultanéité liée aux salles.

Périodes communes et continuité

- Placer chaque pré-soutenance dans un jour de predefence_days et chaque finale dans un jour de final_defence_days. Ces deux listes sont communes à tous les étudiants de l’instance. Elles sont non vides et disjointes ; tous les jours de pré-soutenance précèdent tous les jours de finale.
- Conserver exactement le même président et le même examinateur entre les deux phases d’un étudiant. Les rôles ne sont pas interchangeables. Cette continuité est obligatoire.

Seuls les jours listés sont autorisés pour la phase concernée ; ils ne sont pas nécessairement consécutifs. Le calendrier commun assure l’ordre des deux phases, sans délai individuel à calculer entre les passages d’un étudiant. Les disponibilités des personnes restent obligatoires dans les deux périodes ; les contraintes de salle concernent uniquement les finales.

Une forte concentration des jurys ou une faible attente ne rend pas un planning invalide acceptable. Distinguez toujours la faisabilité des préférences d’organisation décrites à la page suivante.

3. Mesurer la qualité d’un planning

Le validateur calcule cinq critères pour les solutions valides uniquement. Tous sont à minimiser. Présentez toujours le vecteur complet dans l’ordre ci-dessous, quel que soit votre objectif d’optimisation.

Jours de mobilisation des enseignants

teacher_presence_days additionne les jours où chaque enseignant participe à au moins une séance, en ligne ou en salle. Plusieurs séances le même jour ne comptent qu’une fois par enseignant ; trois enseignants mobilisés ce jour comptent pour trois.

Créneaux d’attente des enseignants

teacher_idle_slots additionne, pour chaque enseignant et jour occupé : dernier créneau − premier créneau + 1 − nombre de séances. Une seule séance dans la journée donne zéro. Les intervalles entre deux journées ne sont pas comptés.

Exemple : des séances aux créneaux 1 et 4 laissent deux créneaux d’attente, les créneaux 2 et 3.

Écart de charge entre enseignants

teacher_load_range est le nombre maximal de séances suivies par un enseignant moins le minimum, en incluant tous les enseignants, même inutilisés. Présentez aussi les charges individuelles renvoyées par le validateur.

Dispersion des étudiants d’un même encadrant

supervisor_group_breaks compte des groupes supplémentaires par encadrant et phase, selon les règles suivantes.

En pré-soutenance, relever pour chaque encadrant et jour les créneaux distincts de ses étudiants. Un groupe est une suite maximale de créneaux consécutifs. Des passages simultanés comptent pour un seul créneau. Exemple : les créneaux 1, 1, 2, 4 forment deux groupes, {1, 2} et {4}. Aucune salle fictive n’intervient.

En finale, trier les séances par jour, salle et créneau. Un groupe réunit des séances contiguës d’étudiants du même encadrant. Un changement de salle, de jour, d’encadrant ou un créneau vide rompt le groupe.

Pour chaque encadrant et phase, totaliser les groupes de tous les jours et, en finale, de toutes les salles. Additionner max(0, total − 1). Ce critère n’impose pas la présence de l’encadrant aux pré-soutenances.

Réutilisation des couples de jury

distinct_jury_pairs compte les couples ordonnés distincts (président, examinateur) attribués aux pré-soutenances. Un couple réutilisé ne compte qu’une fois, quels que soient les jours et espacements. Inverser les rôles crée un autre couple.

La réutilisation entre étudiants est un objectif souple ; la continuité entre les deux phases d’un étudiant reste obligatoire.

Arbitrer entre les critères

Justifiez une priorité lexicographique, une combinaison pondérée avec des échelles explicites, ou une démarche de Pareto. Aucune somme pondérée n’est imposée. Rendez visibles les améliorations et dégradations de chaque critère.

4. Concevoir le modèle et les méthodes

Vous êtes responsables de la formulation du problème et des algorithmes. Le dossier fournit des données, un contrôle de conformité et un outil de lecture des plannings. Il ne fournit pas de méthode de résolution à reproduire.

Construire votre modèle

- Définir les données utilisées, les décisions à prendre et leur représentation : affectations de jurys, jours, créneaux et salles pour les finales uniquement. Expliquer comment les décisions des deux phases sont liées.
- Traduire chaque contrainte impérative sans omission. Relier les règles métier à votre modèle et aux tests qui les vérifient. Préciser les hypothèses de votre programme ; elles ne doivent pas modifier les règles du sujet.
- Formaliser les critères retenus et la stratégie multiobjectif. Distinguer une valeur calculée par votre modèle d'une valeur recalculée sur le planning par le validateur.
- Discuter la taille de la représentation, les difficultés combinatoires et les choix permettant de traiter les grandes instances.

Développer une méthode de référence

Implémenter une construction simple et explicable, par exemple une procédure gloutonne avec un ordre de traitement défini et un choix du premier placement compatible. Elle doit rechercher réellement la faisabilité, vérifier toutes les contraintes et produire le même format de sortie que la méthode d'optimisation.

Vous pouvez ajouter une réparation limitée ou un retour arrière. Décrivez-les précisément : le lecteur doit comprendre ce qui rend la méthode simple et pouvoir la reproduire. Une référence volontairement dégradée ne permet pas une comparaison utile.

Développer au moins une méthode d'optimisation

Choisir une approche adaptée : programmation en nombres entiers, programmation par contraintes, recherche locale, métaheuristique ou combinaison de méthodes. Justifier le choix par les décisions, les contraintes et les objectifs du problème. Aucun algorithme particulier n'est obligatoire.

Détailler ce qui est propre à votre approche : formulation, voisinages, construction initiale, gestion des violations pendant la recherche, réparation, paramètres et condition d'arrêt. Une pénalisation interne est possible ; seul un planning respectant toutes les contraintes peut être livré comme solution valide.

Soigner la réalisation

- Séparer lecture des données, représentation du problème, résolution, export et analyse. Ne pas coder de règle spécifique à un identifiant ou à une instance particulière.
- Tester les cas limites : rôle incompatible, domaine incorrect, chevauchement, indisponibilité, plafond journalier, jour hors de sa période, salle attribuée à une pré-soutenance et rupture de continuité.
- Conserver la meilleure solution valide trouvée pendant la recherche et l'exporter à l'arrêt. Vérifier le fichier exporté, pas seulement un objet intermédiaire en mémoire.

Les bibliothèques d'optimisation sont autorisées. Identifiez les dépendances et leurs versions ; distinguez votre contribution des outils et des ressources externes utilisés.

5. Mener une comparaison reproductible

Appliquer le même protocole à la méthode de référence et à chaque méthode d’optimisation, sur les 30 instances. La comparaison doit permettre de distinguer faisabilité, qualité obtenue et coût de calcul.

Budget commun et conditions de calcul

- Limiter chaque exécution à 60 secondes de temps mural par méthode et par instance. Le budget commence après le chargement des données et comprend la construction du modèle, la construction d’une solution initiale et toute la recherche.
- Mesurer séparément la lecture des fichiers, l’export, la validation et la visualisation, qui restent hors budget de résolution. Une solution initiale réutilisée par une méthode fait partie de son budget.
- Utiliser la même machine et une même limite de threads pour comparer les méthodes. Déclarer processeur, mémoire, système, langage, versions des dépendances et nombre de threads.
- Exécuter chaque méthode stochastique avec les graines 0, 1, 2, 3 et 4 sur chaque instance. Une méthode déterministe requiert une exécution par instance. Documenter toute source d’aléa, y compris celle du solveur.
- Figer les paramètres avant la campagne finale. Expliquer comment ils ont été choisis et distinguer les essais de réglage des résultats finaux. Conserver commandes, configurations et journaux.

Données à consigner pour chaque exécution

Enregistrer l’identifiant de l’instance, la méthode, la graine ou la mention déterministe, le temps de construction, le temps de recherche, le temps total de résolution, le statut d’arrêt et le résultat du validateur. Ajouter les cinq métriques pour chaque planning valide, ainsi que le chemin du fichier de solution et du rapport de validation.

Consigner aussi, si disponible, le temps jusqu’à la première solution valide. Un dépassement de budget, une erreur ou une absence de solution doit rester visible dans les résultats ; ne retirez pas ces exécutions du bilan.

Analyser les résultats

- Donner le taux d’exécutions valides et le nombre d’instances avec au moins une solution valide, pour chaque méthode et chaque taille.
- Pour chaque méthode d’optimisation, comparer les critères à ceux de la méthode de référence sur les instances où toutes deux ont une solution valide. Indiquer le nombre de cas concernés. Ne pas mélanger une pénalité d’échec et une métrique de planning.
- Pour les méthodes stochastiques, présenter au minimum médiane et étendue des temps et des cinq critères, ainsi que le nombre de répétitions valides. Montrer les compromis, les régressions et les différences selon la taille.
- Présenter au moins deux plannings commentés avec le visualiseur, dont un cas où une amélioration de qualité est observable, si votre campagne en produit.

Employer des conclusions justes

Une limite de temps atteinte sans solution signifie « aucune solution trouvée dans le budget ». Elle ne prouve pas que l’instance est irréalisable. Une solution améliorée n’est pas nécessairement optimale. Une affirmation d’optimalité ou d’irréalisabilité doit être accompagnée d’une preuve ou d’un certificat explicite relatif au modèle étudié.

6. Utiliser le dossier et livrer le travail

Prendre en main les fichiers

Lire README.md avant de lancer les outils. Le manifeste data/manifest.json répertorie les 30 instances. Le dossier examples contient une petite démonstration, sa solution et un squelette de sortie ; la démonstration ne fait pas partie du benchmark et le squelette vide n'est pas une solution valide.

Chaque instance JSON décrit les domaines, étudiants, enseignants, salles, disponibilités, plafonds et périodes communes predefence_days et final_defence_days. Les jours et créneaux sont numérotés à partir de zéro. Chaque passage occupe un seul créneau. Conserver exactement les identifiants.

Exporter une solution conforme

Le JSON de sortie contient schema_version (« 4.0 »), instance_id et sessions. Une séance comporte student_id, phase, day, slot, room_id, president_id et examiner_id. phase vaut predefence ou final. room_id vaut null en pré-soutenance et l'identifiant d'une salle physique en finale. L'encadrant est déduit des données.

Respecter le schéma du README : types, champs requis et absence de champs supplémentaires. Écrire les entiers JSON sans décimale ni exposant : 1 est accepté, 1.0 et 1e0 sont refusés. Conserver les journaux, paramètres et statistiques hors du JSON de solution.

Valider et visualiser

Depuis le dossier étudiant, exécuter :

python validator.py data/small_01.json submission.json

Le validateur renvoie les erreurs ou les cinq métriques : code de sortie 0 si la solution est valide, 1 sinon. Il requiert Python 3.10 ou ultérieur, sans paquet externe. Valider une instance seule vérifie sa structure, sans prouver sa faisabilité.

Ouvrir visualiseur.html dans un navigateur, par un double-clic, sans serveur. Importer l'instance puis la solution pour lire les horaires, salles et jurys. L'inspection visuelle complète le contrôle automatique. En cas de divergence, le validateur Python fait foi ; consulter le README pour les détails.

Contenu de la remise par groupe

Livrable	Contenu attendu
Code et notice	Sources de la méthode de référence et de chaque méthode d'optimisation, dépendances, versions et commandes pour reproduire un cas puis la campagne complète.
Rapport de 8 à 12 pages	Problème et modèle, choix des méthodes, protocole, résultats comparés, analyse des échecs et limites. Annexes techniques possibles.
Résultats reproductibles	Une ligne par exécution, configurations et graines, journaux, fichiers JSON des solutions trouvées et rapports du validateur.
Support de restitution	Présentation synthétique de la méthode et des conclusions, avec deux vues de plannings commentées.
Bilan collectif	Répartition des contributions et références des outils, bibliothèques, contenus et aides utilisés.

7. Organiser le projet et préparer l’évaluation

Les jalons ci-dessous organisent votre travail de développement. Ils sont distincts des phases de pré-soutenance et de soutenance finale présentes dans les données à planifier.

Jalons de travail

Jalon	Résultat vérifiable
1. Compréhension	Lecture du schéma, liste des contraintes, explication des cinq critères et validation de la démonstration.
2. Modélisation	Représentation des décisions, traduction des contraintes et choix argumenté de l’approche multiobjectif.
3. Référence	Méthode simple exécutable, export conforme et premiers tests de faisabilité.
4. Optimisation	Méthode d’amélioration, tests ciblés, paramètres stabilisés et comparaison sur plusieurs tailles.
5. Expérimentation	Campagne complète, fichiers conservés, contrôle de validité et tableaux ou graphiques comparatifs.
6. Restitution	Rapport, code reproductible, analyse des limites, plannings commentés et bilan des contributions.

Barème sur 20 points

Critère	Points	Attendus principaux
Modélisation et contraintes	5	Décisions explicites, contraintes complètes et cohérence avec les données.
Méthodes et optimisation	4	Référence pertinente, méthode justifiée, contribution identifiable et arbitrages expliqués.
Expérimentation et analyse	5	30 instances, protocole commun, validation préalable, résultats complets et conclusions étayées.
Qualité et reproductibilité	3	Code structuré, tests, exports conformes, dépendances et commandes exploitables.
Rapport et restitution	3	Clarté, visualisations utiles, limites comprises et contributions de chacun explicites.

La note ne dépend pas seulement de la meilleure valeur obtenue. Elle porte aussi sur la correction, la qualité du raisonnement, la rigueur des comparaisons et la capacité à expliquer les résultats. Un planning invalide ne peut pas être présenté comme une amélioration.